The Choose-Your-Own-Adventure Calculus

Anonymous Author(s)

Abstract

Some of the most remarkable results in mathematics reveal connections between different branches of the discipline. The aim of this paper is to point out a modest, but still remarkable, similarity between a range of different interactive programming systems. In many programming systems, the user can interactively construct a program by repeatedly triggering some kind of completion mechanism and choosing one of the offered options. This is the case with code editors for object-oriented languages (choosing object members), data exploration environments (choosing an applicable transformation), but also theorem provers (choosing an applicable tactic) and structure editors (choosing a grammar rule).

In this paper, we formally capture the essence of this interaction pattern through a small formal model called *the choose-your-own-adventure calculus*. We show how a wide range of different systems fits the model. Looking at the examples through a common perspective reveals multiple subtle differences. To formally capture those, we characterise basic properties of choose-your-own-adventure systems, resembling those from other areas of programming language research, including correctness, completeness and uniqueness. We further show how the choose-your-own-adventure calculus can be used as the basis for formally studying more advanced interaction patterns including mixed-initiative interaction, AI-based programming assistants and programming by demonstration.

We strongly believe that interaction with programming systems deserves as much attention as the underlying programming languages. Our work is one step in this direction. It provides a way of talking about commonalities and subtle differences between multiple interactive programming systems and enables a transfer of ideas between interactive systems from very different domains.

CCS Concepts: • **Software and its engineering → General programming languages**.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference 'XX, Month DD–DD, 2025, City, Country
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2025/06
https://doi.org/XXXXXXX.XXXXXXX

Keywords: Interactive programming systems, Type providers, Proof assistants

ACM Reference Format:
Anonymous Author(s). 2025. The Choose-Your-Own-Adventure Calculus. In *Proceedings of Conference Title (Conference 'XX)*. ACM, New York, NY, USA, 18 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

TODO: Add argument for why formalism is needed: give a concrete example (e.g. the uniqueness/reconstructibility gap) that informal reasoning would miss.

Multiple interactive programming systems, ranging from code editors for object-oriented programming languages to data exploration systems, interactive proof assistants and structure editors, exhibit a remarkably similar pattern of interaction. They offer the user, who can be a programmer, a data scientist or a proof writer, a range of choices that the user can select from in order to complete their program, script or proof. The user can initiate the interaction iteratively, using it to create and refine a larger part of their program.

There are subtle differences between different implementations of the general pattern. In some systems, the resulting source code will contain a trace of the choices made by the user. For example, when choosing an item from a list of class members, the code will contain the member name. In some systems, the interaction results in a block of code that can be included in the source file, but does not include a trace of the interaction. For example, invoking a proof search or case split in Idris [9] constructs a well-typed program, but leaves no trace of the command used to construct it. The nature of the generated options also varies. The list of choices may include all possible options that are valid at a given location, or it may list only a subset of the valid options. In some cases, it may even include incorrect options as, for example, in auto-completion for dynamic languages [14].

The aim of this paper is to formally capture the recurring interaction pattern:

1. We motivate the formalism by reviewing five different systems that implement a variation on the interaction pattern. These include type providers in F# [51], type providers for data exploration in The Gamma [34, 37], AI assistants for semi-automated data wrangling [40], tooling for interactive proof assistants [4, 9, 54] and structure editing [7] (Section 2).
2. We introduce the *choose-your-own-adventure calculus*, which is a small formal structure that models an interactive system where a user constructs a program by

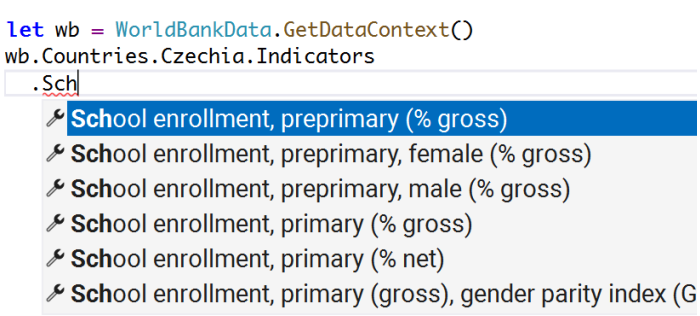

Figure 1. F# code editor showing completions offered by the World Bank type provider.

Figure 2. Constructing a query in The Gamma. We count the number of gold and silver medals for each athlete and sort the data by the number of gold medals.

repeatedly choosing from a list of options offered by the system (Section 3).

3. The calculus allows us to make the aforementioned subtle differences precise. We define the notions of *correctness*, *completeness* and *uniqueness* for the choose-your-own-adventure calculus (Section 5). To understand the way in which the interaction is integrated in a programming environment, we formally define the notion of *expression completion* and capture a specfic kind of *reconstructible* completion (Section 3).
4. We show that multiple programmer assistance tools, such as mixed-initiative interaction, AI-based assistants and programming by demonstration can be defined on top of the primitives offered by the calculus (Section 6). This also illustrates how the choose-your-own-adventure calculus supports transfer of ideas across different kinds of interactive programming systems.

The main contribution of this paper is conceptual rather than technical. We capture a pattern that is perhaps not surprising in retrospect, but that is easy to overlook until it is given a name. We use formal programming language theory methods to precisely describe interesting aspects of the pattern. Moreover, our work also confirms that programming language theory methods can be extremely effective for studying not just *programming languages*, but also interactive *programming systems* [20].

2 Motivation

Computer scientists studying programming have long focused on programming languages as syntactic entities, sometimes neglecting the interactive environments in which they are inevitably embedded [15]. Notably, in many of the motivating examples that we draw from in this section, the interactive aspect of the system is only described in supplementary materials [4, 9, 51]. Only recently, programming language theory started to be used to study interactive environments [1, 28]. Our work contributes to this research direction.

We start by reviewing five instances of the interaction pattern.

TODO: Clarify the two Gamma models upfront at the start of Section 4: Sec. 4.1 generates direct operation calls; Sec. 4.2 models the actual Gamma, which hides operations behind member accesses. In all of them, an editor offers the user a completion list to choose from during working with the system.

Type providers. F# type providers [51] are a mechanism for integrating (primarily) external data sources into the F# type system. A type provider is a compiler extension, loaded and executed at compile-time and at edit-time. It can run arbitrary code to read the structure of external data and use it to generate a suitable statically-typed representation of the data, typically objects with members. Type providers can, for example, infer the type from a sample JSON [39] or read a database schema.

The example in Figure 1 shows a simple type provider for accessing information from the World Development Indicators database. The provided wb object allows the programmer to access any indicator of any country in the database by choosing an appropriate [Country] and an [Indicator] in a chain of members wb.Countries.[Country].Indicator.[Indicator]. The result is a time series with values for the given indicator and a country. More generally, the example can be seen as a special case of a type provider for slicing n-dimensional data cube [37]. We choose a fixed value for two of the three dimensions (country, indicator, time) and obtain a series indexed by the remaining dimension.

When using the type provider, the user types the first line of code to initialize the type provider and triggers autocompletion by typing wb followed by the dot. The rest of the code is constructed by choosing an option from a list and typing another dot (a pattern light-heartedly called *dot-driven development* by Phil Trelford [43]).

Data exploration. The Gamma [37] is a programmatic data exploration environment for non-programmers. In The Gamma,

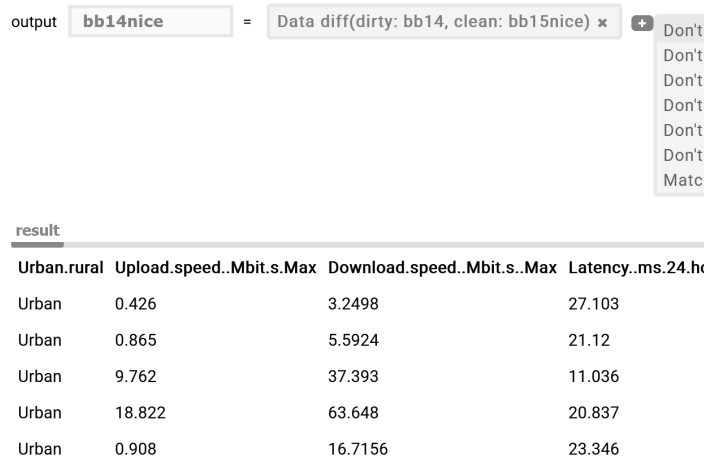

Figure 3. Using the datadiff AI assistant to reconcile the structure of the two datasets. The user is offered a list of constraints to prevent or force matching between specific columns.

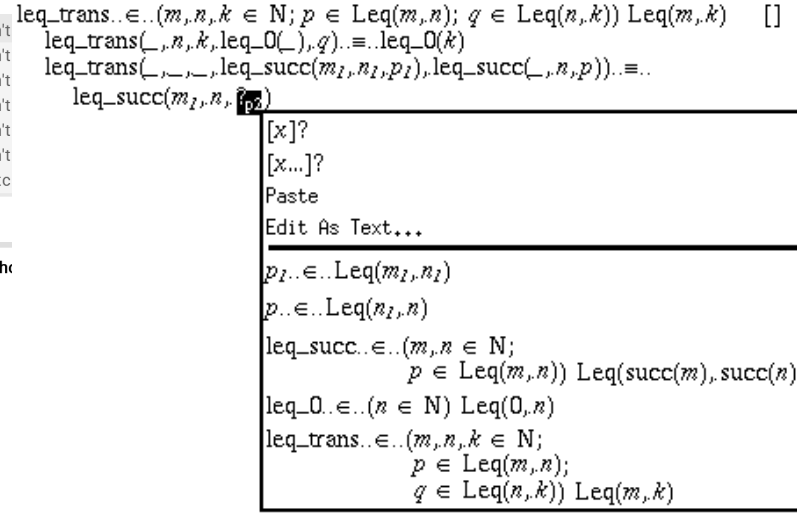

Figure 4. Constructing a proof of the transitivity of the ≤ relation in the Alf editor. The user is offered a range of variables and constructors in scope at the current location. [4]

type providers are the primary programming mechanism. They are used not just for data access, but also for constructing queries.

The type provider shown in Figure 2 lets the user construct an SQL-like query by repeatedly choosing operations and their parameters [34]. It keeps track of the schema and uses it to generate all possible valid parameters. For example, when sorting data, it generates an object with two members for each columns – one for ascending and one for descending order. Similarly, the grouping operation first offers all columns as possible grouping keys and then lets the user choose from a range of pre-defined aggregations (sum, count, average, concatenate). The system also evaluates the query on the fly, providing a live preview [36].

The interaction pattern is the same as before. After the user triggers auto-completion, they repeatedly select an operation and its parameters to construct a query. One notable difference is that the structure of the generated types is potentially infinite (the user can keep adding further operations) and so the types are generated lazily.

AI assistants. The third instance of the choose-your-own-adventure interaction pattern comes from the work on semi-automatic data wrangling tools known as AI assistants [40]. An AI assistant guides the analyst through a data wrangling problem such as reconciling mismatched datasets, filling missing values or inferring data format and types. An AI assistant solves the problem automatically and suggests an initial data transformation, but it also generates a number of constraints that the user can choose from to refine the initial solution. If the initial solution is not correct, the user chooses a constraint and the AI assistant runs again, suggesting a new data transformation that respects the constraint.

Figure 3 shows an example. It uses the datadiff AI assistant [49], running in a Wrattler notebook [38], to merge broadband quality data published by Ofcom for two subsequent years. The format of the CSV files for the two years differs. Columns were added, removed, renamed and their order has changed. In the example, we selected 6 columns from the year 2015 and want to find matching data from 2014.

When the AI assistants runs automatically, it correctly maps the numerical columns, but it incorrectly maps the Urban.rural (2014) column to Nation (2015). This happens because both columns are categorical and have three values with similar statistical distribution. A data analyst can easily spot the mistake. They click the “+” button to add a constraint and choose Don’t match ‘Urban.rural’ and ‘Nation’ to specify that the two columns should not be matched. Datadiff then runs again and finds the correct matching.

The interaction patter is the same as in the previous two cases. The analyst constructs the correct data transformation by repeatedly choosing from a list of options, until they obtain the desired result. However, the way the interaction pattern is implemented differs. First, in the case of type providers, we are gradually constructing a program by adding operations to a method chain. An AI assistant automatically synthesizes a data transformation (program) and we are gradually adding constraints to control the synthesis. Second, in the case of type providers, the completion list offered all possible members of the object. Now, the list offers constraints recommended by the AI assistant which may not be complete.

Interactive theorem provers. A fourth example of the choose-your-own-adventure interactive pattern can be found in interactive theorem provers. When writing programs in systems like Idris [10], the user typically works by stating the desired conclusion and filling the implementation with a hole. The system provides a range of interactive editing capabilities to fill the holes [29]. It can, for example, generate a case split or search for a proof [9].

Systems like Idris provide key bindings to invoke the completions, but the functionality could also be offered through a user interface. An example that illustrates this is the interactive editor for the Alf theorem prover [25], which is based on a gradual refinement of an incomplete proof object [4]. This is illustrated in Figure 4. The user is proving the transitivity of the $\leq$ relation for Peano arithmetic natural numbers. They pattern match on the proof argument p and complete the first branch. For the second branch, they need to fill a hole $?_{p2}$ (called a wildcard in Alf). They trigger a completion and a pop-up menu shows the available variables and constructors, including leq_trans that can be used to complete the proof. After choosing leq_trans, two new holes are generated for its arguments. Those can be, again, filled interactively, by choosing p_1 and p from the completion.

The interaction pattern is again the same. The user repeatedly triggers a completion and uses it to refine and complete a proof by filling holes. There are subtle differences too. Unlike earlier, each completion directly refines the proof that the user is editing; a completion may also generate multiple new holes, rather than just appending to a chain of operations.

The Alf editor is a historical example, but a similar user interface could be built for systems like Idris or Coq. The two would work differently. As in Alf, Idris source code represents the proof itself and a completion would replace a hole with a suggested term. In Coq, the proof is a series of tactic invocations and so selected completions would be added to this list and would form a trace of the interaction with the user.

Structure editors. Structure editors make it possible to construct programs by manipulating the abstract syntax tree of a program rather than by working with text. The interaction with a structure editor can be based on menus [52], key bindings [55], tiles [30] and a range of other approaches. A number of those fit the choose-your-own-adventure pattern.

Sandblocks [7] is an example of a recent structure editor that is interesting for multiple reasons. First, the editor is automatically generated from a grammar. Second, the editor combines keyboard input with context menus (Figure 5). In the above example, the user is typing code and needs to fill a placeholder for an expression. After they type the start of the expression, the editor offers a completion list with possible production rules from the grammar that are valid in the given context and are compatible with the text typed so far. The user can continue typing (to further refine the recommendation), but they can also select a rule from

Figure 5. Constructing a program using the Sandblocks structure editor [7]. The user is typing the start of an expression and context menu offers possible production rules according to a grammar.

the menu. This completes the expression according to the production rule, generating further empty holes that can, in turn, be interactively filled with code.

In Sandblocks, context menus serve mainly as hints and users of the editor often construct programs through typing, but the system illustrates the fact that structure editors can also be based on the interaction pattern captured in this paper. The editor can offer all possible production rules based on the given grammar and the user could construct an entire program by choosing one of the rules (and manually entering terminals such as strings and identifiers).

3 Formal model

A system that implements the choose-your-own-adventure interaction pattern repeatedly offers the user a range of options to choose from. Each of the options is designated by an identifier. The system also maintains a state during the process which determines subsequent options. The state may not be visible to the user, but the user can always explicitly request the program constructed so far.

We can think of the interaction with the system as navigating through a tree structure, starting from a root and choosing one of the possible branches in each step. In the following definition, the key choices operation can thus be seen as returning branches of a given node.

Definition 3.1 (Choose-your-own-adventure system)**.** Given expressions $e \in \mathbb{E}$ and states $\sigma \in \Sigma$, a choose-your-own-adventure system is a pair of operations choices, choose such that:

- $\text{choices}(\sigma) = \{\iota_1 \mapsto \sigma_1, \ldots, \iota_n \mapsto \sigma_n\}$ is an operation that takes a state and
  generates options designated by an identifier ι_i and represented by a state σ_i,

- $\mathsf{choose}(\sigma) = e$ is an operation that returns generated program for a given state.

The definition is not a programming language calculus in the usual sense in that it does not define a concrete syntax with reduction rules. It is an abstract algebraic description of the structure of a system that supports the choose-your-own-adventure interaction pattern. The definition is close to that of an AI assistant [40], which is written using a language specific for the data wrangling domain (such as cleaning scripts or input and output data) but is structurally similar. It is also worth noting that the definition may describe not just trees, but also graphs with cycles. A system where the user can return to a previously visited state may not be practically useful, but it does not pose a theoretical problem.

Expression completion. One of the subtle questions about the choose-your-own-adventure pattern raised in the introduction is whether a trace of the interaction can be reconstructed from the source code of an interactively constructed program. That is, if we follow a sequence of choices $\iota_1, \ldots, \iota_n$ to construct a program e, is it possible to recover the original sequene**TODO: typo: sequene→sequence** of choices through which it was constructed just from the program e.

The choose-your-own-adventure interaction pattern is typically used to complete a partial program. To model this, we assume that the host language has a notion of a hole, written as ? and that a user can select a part of program to invoke the completion on. When invoked on an expression containing multiple holes, the system can start by completing the first hole.

We write $E[e]$ for a completion context, akin to evaluation contexts in operational semantics. We assume that, for a program containing a hole in a completion context $E[?]$, we can construct an initial choose-your-own-adventure state using an operation $\mathsf{init}(E[?]) = \sigma_0$.

Definition 3.2 (Expression completion)**.** An expression $E[?]$ is completed as $E[e]$ through an interaction with a choose-your-own-adventure system consisting of choices and choose if:

1. $\mathsf{init}(E[?]) = \sigma_0$ obtains the initial state of a choose-your-own-interaction system,
2. σ_n is a system state such that $\forall i \in 1 \ldots n \,.\, (\iota_i \mapsto \sigma_i) \in \mathsf{choices}(\sigma_{n-1})$, i.e., the user makes a series of choices resulting in a final state of the system σ_n,
3. $E[e]$ where $e = \mathsf{choose}(\sigma_n)$ is the final program constructed by replacing the hole in the completion context with the expression e generated from σ_n.

As we will see when we revisit the earlier examples formally, this way of invoking a choose-your-own-adventure system is used, for example, in the case of interactive theorem provers. In those systems, the user triggers the completion on a proof (program) containing a hole. They then fill the hole and, possibly iteratively, further holes in the generated proof. The final expression is embedded in the source code, but it does not indicate what options, identified by $\iota_1, \ldots, \iota_n$, were selected in the process.

Reconstructible completion. In many choose-your-own-adventure systems, it is possible to reconstruct a trace of the interaction through which a program was constructed. This is the case with type providers, where the user chooses a sequence of object members to be accessed and those members directly appear in the source code. The same would be the case in a completion system for Coq that would offer tactics to apply, becuase**TODO: typo: becuase→because** the resulting proof would consist of a sequence formed by the selected tactics.

For systems such as type providers, we say that that expression completion is *reconstructible*. To capture the notion formally, we again need the init operation, but also an operation decode that extracts identifiers of invoked completions from an expression:

Definition 3.3 (Reconstructible expression completion)**.** An expression $E[?]$ is reconstructibly completed as $E[e]$ through an interaction with a choose-your-own-adventure system consisting of choices and choose if:

1. $E[?]$ is completed as $E[e]$ using init, choices and choose
   through a series of choices designated by identifiers $\iota_1, \ldots, \iota_n$,
2. it also holds that $\mathsf{decode}(e) = (\iota_1, \ldots, \iota_n)$.

If the integration of a choose-your-own-adventure system with a host programming language uses reconstructible expression completion, we can recover the choices through which the user constructed the expression e in a completion context $E[e]$. This also means that we can reconstruct the final state σ_n of the system by starting from $\mathsf{init}(E[?])$ and following the choices specified by $\iota_1, \ldots, \iota_n$.

4 Examples

We now revisit the five examples from Section 2 and show how they fit the above formal model. All five examples rely on some domain-specific logic. We describe what information the logic provides, but refer to full description elsewhere for details.

To show how the model lets us distinguish subtle details of interactive programming systems, we start with a model of data exploration system that is inspired by The Gamma, but differs in one notable way. We then discuss type providers more generally and show how to model The Gamma more precisely. We then revisit the remaining three examples.

4.1 Data exploration

In The Gamma, the choose-your-own-adventure interaction pattern is used to construct a query that transforms the given input data. The query is a sequence of operations with

parameters, $op(p_1, \ldots, p_n)$, loosely modelled after relational algebra [12].

In The Gamma, the query is hidden from the data analyst. Behind the scenes, the system generates objects with members and the identifiers designating individual options are the names of those members. The operation is encapsulated in the code of the accessor of the member. In the simplified model developed in this section we ignore this fact. The model presented here directly generates code that calls the underlying operations. For example, assume that the user makes the following choices:

«group data» . «by Athlete» . «sum Gold» . «count all» . «then»

In The Gamma, the individual identifiers become object members and they are included as a member chain in the generated code. In the simplified model presented here, the completion instead fills the hole with an expression representing the operation:

$$\mathsf{group}(\texttt{"Athlete"}, \mathsf{sum}(\texttt{"Gold"}), \mathsf{count}())$$

The two approaches have different human-computer interaction trade-offs. In terms of cognitive dimensions [8, 16], the latter has a greater closeness of mapping, while the former is less cognitively demanding to read for a non-programmer. As we will see in Section 5, the two implementations of the interaction pattern also differ in terms of their formal properties.

Formal model. The options generated by The Gamma let the user select both the next operation and the parameters of the previously selected operation. The available operations and parameters are generated based on a schema S that is transformed by the operations. The state of the system σ contains the current schema S and the operations applied so far. In the following, we write $op(\boldsymbol{p})$ for an operation with a vector of parameters:

$$\sigma = S, [op_1(\boldsymbol{p_1}), \ldots, op_n(\boldsymbol{p_n})]$$

The behaviour of the choices operation depends on whether the last operation in the sequence expects further parameters or whether it is fully-specified. In the first case, the recommendation engine generates possible additional parameter values $p', p'', \ldots$ based on the schema S, the operation op_n and the already known parameters $\boldsymbol{p_n}$. The choices operation then generates options that add the additional parameter. We generate the identifiers $\iota', \iota'', \ldots$ based on the state and the parameter value, such as «by Gold descending». Note that adding a parameter may also result in a new schema $S', S'', \ldots$ (which the recommendation engine computes based on the previous schema and the new parameter):

$$\begin{aligned}&\mathsf{choices}(S, [op_1(\boldsymbol{p_1}), \ldots, op_n(\boldsymbol{p_n})]) = \\ &\quad \{\ \iota' \mapsto (S', [op_1(\boldsymbol{p_1}), \ldots, op_n(\boldsymbol{p_n}, p')]), \\ &\quad\ \ \iota'' \mapsto (S'', [op_1(\boldsymbol{p_1}), \ldots, op_n(\boldsymbol{p_n}, p'')]),\ \ldots\ \}\end{aligned}$$

If the last operation takes no further parameters, the system produces a choice of possible next operations $op', op'', \ldots$. Again, we are also given new schemas $S', S'', \ldots$ and we generate identifiers $\iota', \iota'', \ldots$ based on the operation name. The choices operation then returns options that append the additional operation:

$$\begin{aligned}&\mathsf{choices}(S, (op_1(\boldsymbol{p_1}), \ldots, op_n(\boldsymbol{p_n}))) = \\ &\quad \{\ \iota' \mapsto (S', [op_1(\boldsymbol{p_1}), \ldots, op_n(\boldsymbol{p_n}), op'()]), \\ &\quad\ \ \iota'' \mapsto (S'', [op_1(\boldsymbol{p_1}), \ldots, op_n(\boldsymbol{p_n}), op''()]),\ \ldots\ \}\end{aligned}$$

Finally, the choose operation takes the state σ and generates an expression that represents the data transformation. This is only possible if all parameters are fully-specified. For simplicity, assume that k is the index of the last fully-specified operation (either n or $n-1$). If the host language lets us compose functions using $f \circ g$, we can write:

$$\mathsf{choose}(S, (op_1(\boldsymbol{p_1}), \ldots, op_n(\boldsymbol{p_n}))) = op_1(\boldsymbol{p_1}) \circ \ldots \circ op_k(\boldsymbol{p_k})$$

The recommendation engine behind The Gamma provides a domain-specific logic for generating possible operations and their parameters based on the current schema of the data. As the above definition shows, this underlying engine can be wrapped and exposed through the common choose-your-own-adventure interface.

4.2 Type providers

The type provider mechanism in F# operates at the level of the type system. It is not merely an editor feature. A type provider for a data source, such as the World Development Indicators database, generates a collection of types that model the external data source. In F#, the types are classes with members that implement the logic to retrieve data at runtime.

The auto-completion mechanism in F# code editors, which implements the choose-your-own-adventure interaction pattern, is not specific to type providers. It offers a list of members of an object based on its type. We model the completion as an iterative process, repeatedly adding further members to a chain. The state σ thus consists of an initial expression on which the completion is invoked, a chain of selected members and the type of the last member.

To model the completion mechanism, we also need a representation of the type information. We loosely follow the Foo calculus model [39] and write $\mathbb{C}$ for a set of class definitions, each consisting of an implicit class constructor and a collection of members M:

$$\begin{aligned}\sigma &= e.\iota_1.[\ldots].\iota_n, C \\ \mathbb{C} &= \{C \mapsto \mathsf{type}\ C(\overline{x:\tau}) = \overline{M},\ \ldots\ \} \\ M &= \mathsf{member}\ \iota{:}C = e\end{aligned}$$

Each member in the Foo calculus consists of a of a name ι, return type C and implementation e. For our purposes, we only need the type information and the operations that define the choose-your-own-adventure are parameterized by the set of classes $\mathbb{C}$.

The $\mathsf{choices}_{\mathbb{C}}$ operation finds the class definition corresponding to the type of the last member in the current chain.

It generates choices appending each of the available members to the current chain. The $\text{choose}_{\mathbb{C}}$ operation returns the constructed member chain:

$$\begin{array}{rcl} \text{choices}_{\mathbb{C}}(e.\iota_1.[\ldots].\iota_n, C) & = & \\ \{\ \iota' \mapsto (e.\iota_1.[\ldots].\iota_n.\iota', C') & & \\ \iota'' \mapsto (e.\iota_1.[\ldots].\iota_n.\iota'', C''),\ \ldots\ \} & & \\ \text{where } \mathbb{C}(C) & = & \text{type } C(\overline{x:\tau}) = M', M'', \ldots \\ \text{and } M' & = & \text{member } \iota' : C' = e' \end{array}$$

$$\text{choose}_{\mathbb{C}}(e.\iota_1.[\ldots].\iota_n, C) = e.\iota_1.[\ldots].\iota_n$$

The model does not directly refer to type providers. Those are responsible solely for generating the type definitions in $\mathbb{C}$ as documented in earlier work [39]. It is worth noting that the type provider for data exploration, implemented by The Gamma, additionally needs to generate classes lazily [34]. To model this aspect, the simple lookup $\mathbb{C}(C)$ needs to be replaced with an operation that returns the type definition, alongside with a new context $\mathbb{C}'$ that contains additional generated type definitions (return types for all the members of the class C).

The model follows the reconstructible expression completion model. It is easy to define the decode operation that takes the resulting generated expression and returns the sequence of choices, because the choices are items of the member chain. **TODO: Sketch decode: e.g. decode**$(e.\iota_1.\ldots.\iota_n) = (\iota_1, \ldots, \iota_n)$ **— don't just assert it's easy.** A slight caveat is that the completion is not invoked on an empty hole, but on a hole that contains the initial expression on which the completion is applied. We can model this using filled holes [32] and write $?_e$ for a hole containing the initial expression e. The $\text{init}(?_e)$ operation then returns e alongside with an empty chain and the type of e.

As noted earlier, The Gamma does not embed query expressions directly into the generated code. It uses the model presented in this section and generates choices as members of types behind the scenes. We return to the differences between the two models in Section 5.

4.3 AI assistants

AI assistants guide the analyst through a data wrangling task. They generate a data cleaning script, taking into account constraints selected by the user. Most AI assistants obtain the script by performing statistical optimization with respect to a set of constraints specified by the user. That is, they look for an expression from the set of all possible expressions that optimizes some objective function that assigns score to the expression with respect to the given input data. Note that AI assistants do not iterate over all possible expressions. They use a machine learning method to approximate a solution to the problem.

An optimization-based AI assistant [40] thus provides another, very different, way of implementing the choose-your-own-adventure pattern. The assistant operates with respect to some input data X that do not change during the interaction and so we parameterize the choose-your-own-adventure calculus operations by the data. The input data X can be actual input data or a representative sample and so the AI assistant can use past data to infer a cleaning script that will be used on new inputs.

The state σ consists of a set of constraints specified by the user. We write c for individual constraints and $\boldsymbol{c}$ for a set of constraints. The initial state is an empty set:

$$\begin{array}{rcl} \sigma & = & \{c_1, \ldots, c_n\} \\ \sigma_0 & = & \emptyset \end{array}$$

Unlike in the previous examples, the crucial logic of an AI assistants is implemented in the choose operation. The operation runs the optimization algorithm to choose the best cleaning script for given constraints. Formally, this can be written using the arg max operator which finds an argument (an expression) for which the given function (scoring function) is maximized. The user-specified constraints can either restrict the set of possible expressions or influence the scoring function. More formally, we assume that:

- $E_{\boldsymbol{c}} \subseteq E$ is a set of expressions that respect constraints $\boldsymbol{c}$,
- $Q_{\boldsymbol{c}}(X, e)$ is a scoring function with respect to the constraints $\boldsymbol{c}$, which returns the score of an expression e, i.e., how good e is at cleaning the data X.

For a given set of constraints $\boldsymbol{c}$, the choose operation looks for $e \in E_{\boldsymbol{c}}$ with the largest score:

$$\text{choose}_X(\boldsymbol{c}) = \arg\max_{e \in E_{\boldsymbol{c}}} Q_{\boldsymbol{c}}(X, e)$$

The actual implementation of the optimization uses various machine learning techniques to find the optimal expression. In case of datadiff, X is a pair of datasets X_1, X_2 to be reconciled. The AI assistant uses the Hungarian algorithm [49] to construct a matching of columns from X_1 and X_2. The generated expression is a sequence of patches that can be applied to X_2 in order to reconcile its structure with the sturcture**TODO: typo: sturcture→structure** of X_1. The constraints specified by the user restrict the space of possible column matchings and so they affect $E_{\boldsymbol{c}}$. The scoring function $Q_{\boldsymbol{c}}$ is independent of the constraints and computes a sum of distances between the statistical distributions of the columns from X_1 and the patched version of X_2.

The choices operation is responsible for generating possible constraints that the user may want to add to guide the inference. AI assistants typically offer the user options to prevent or adapt some aspect of the cleaning logic inferred by the system. For example, if datadiff matches two columns, it will offer a constraint to prevent the matching. It also generates constraints that let the user force a specific matching.

To implement choices_X, optimization-based AI assistants first call $\text{choose}_X(\sigma)$ to get the best expression e. Based on this, they generate possible constraints $c_1, c_2, \ldots$ that the user may want to choose from. The identifiers $\iota_1, \iota_2, \ldots$ provide a

human-readable description of the constraints. Note that this operation is specific to the particular AI assistant; choices_X then offers a list of constraint sets with one of the additional constraints:

$$\text{choices}_X(\boldsymbol{c}) = \{\iota_1 \mapsto \boldsymbol{c} \cup \{c_1\}, \iota_2 \mapsto \boldsymbol{c} \cup \{c_2\}, \ldots\}$$

The expression completion for an AI assistant, as described here, is not reconstructible. The interaction results in a cleaning script (expression), but there is no way of reconstructing the constrains used to guide the optimization. To make the completion reconstructible, the choose operation would need to explicitly include the constraints in the resulting expression. However, rerunning the choose operation with the same constraints may result in a different cleaning script if the machine learning algorithm is probabilistic. **TODO: Inconsistency:** choose **uses** arg max **(deterministic) but is described here as probabilistic. Either model** choose **as non-deterministic or acknowledge** arg max **is an idealisation.**

4.4 Theorem proving

In the previous three sections, we showed how existing formally well-documented systems fit the choose-your-own-adventure interaction pattern. Although interactive theorem provers and editors for dependently typed languages implement similar kinds of interactions, there is no well-documented system that fits the pattern exactly. The closest example is perhaps the recently envisioned mixed-mode interaction theorem prover [54]. Rather than reframing the implementation of an existing system, this section thus outlines a possible implementation.

TODO: Formalise the Coq tactic-sequence case (state = sequence of tactics) alongside the Idris term-based case — only Idris is formally modelled; Coq is described in prose. TODO: Discuss Programming by Navigation [lubin-2025-pbn]: maps directly onto CYOA (step set = choices, Strong Soundness = Correctness, Strong Completeness = Completeness); note reconstructibility is what CYOA adds.

Theorem provers. There are two approaches to interacting with an interactive theorem prover. In Coq, the user writes a sequence of tactics that transform proof goals. In Agda or Idris, the user writes a term, or program, of a type that represents the theorem. Interactive editors exist for both kinds of systems. For Coq, the `Company-Coq` [5, 41] extension offers auto-completion, which recommends available tactics, hypotheses and local definitions, but it does not filter them based on what is valid in a given context. For Idris, the interactive editor [9] offers a range of commands that transform the selected term by adding a case split, a missing case or by automatically searching for a proof. In Idris, the system produces valid completions, but those cover only a small number of situations.

$$\frac{}{\Gamma, x : \tau \vdash \tau \Rightarrow x}\ \text{(syn-var)} \qquad \frac{}{\Gamma \vdash \tau \Rightarrow ?_\tau}\ \text{(syn-hole)}$$

$$\frac{\Gamma, x : \tau_1 \vdash \tau_2 \Rightarrow e}{\Gamma \vdash \tau_1 \to \tau_2 \Rightarrow \lambda x.e}\ \text{(syn-lambda)} \qquad \frac{\Gamma \vdash \tau_1 \to \tau_2 \Rightarrow e_1 \quad \Gamma \vdash \tau_1 \Rightarrow}{\Gamma \vdash \tau_2 \Rightarrow e_1\ e_2}$$

$$\frac{b \in \{\text{true}, \text{false}\}}{\Gamma \vdash \text{bool} \Rightarrow b}\ \text{(syn-bool)} \qquad \frac{\Gamma \vdash \tau \Rightarrow e_1 \quad \Gamma \vdash \tau \Rightarrow e_2 \quad \Gamma \vdash}{\Gamma \vdash \tau \Rightarrow \text{if } e \text{ then } e_1 \text{ els}}$$

Figure 6. Illustrative set of simple type-directed program synthesis rules

Despite the different ways of working, an implementation of the choose-your-own-adventure pattern for both kinds of systems would be similar. Based on the current sub-goal, the system would recommend a range of tactics that can be applied to the sub-goal. In the case of Coq, the selected tactic would be added to the sequence. In Idris, the selected tactic would be applied to transform the current term. The difference is in the expression completion. A system for Coq would provide reconstructible completion in that the selected option is added to the proof source code. (Much like selecting a completion when using type providers appends a member access.) A system for Idris or Agda would use the tactic to transform the term, making it impossible to reconstruct the sequence of applied tactics.

Type-directed synthesis. Thanks to the equivalence between programs and proofs, techniques akin to tactic-based proof construction have also emerged in work on type-directed program synthesis [22]. As illustrated in Figure 6, the synthesis process can be described as a set of rules of the form $\Gamma \vdash \tau \Rightarrow e$ that describe ways of synthesizing expressions e of a type τ. Existing implementations of the mechanism typically aim to automate program synthesis and use more precise type information, such as refinement types [42] and graded types [19], or include examples [33]. However, the same rules could be used to guide an interactive choose-your-own-adventure system. If the interaction was invoked to fill a typed hole $?_\tau$ in a context Γ, the system could collect all expressions e such that $\Gamma \vdash \tau \Rightarrow e$ and offer a choice of those options. Note that the definition in Figure 6 synthesizes sub-expressions recursively, but a choose-your-own-adventure system may proceed one step at a time, filling sub-expressions with typed holes using (syn-hole).

Formal model. From the perspective of user interaction, a proof assistant where a user interactively constructs a term of a given type is very similar to an interactive tool for type-directed program synthesis. The key difference being that theorem provers like Idris and Agda use rich dependent type theories.

For example, consider a system akin to Idris where the user aims to construct a term e of type τ. The term may contain typed holes written as $?_\tau$ and a relation $\Gamma \vdash \tau \Rightarrow e$ provides ways of synthesizing terms of type τ. We again write

$E[?_\tau]$ for a completion context containing a (typed) hole; we assume that the variables Γ available in the completion context of the hole can be obtained using $\mathsf{vars}(E[?_\tau])$.

To model a choose-your-own-adventure interaction akin to Idris, the state of the system would be the term e itself, initially a typed hole. The choices operation synthesizes possible completions using $\Rightarrow$ (restricted, e.g., to only generate terms of a certain maximum size) and offers the resulting terms as possible completions. The identifiers ι could be based either on the tactic name (rule name) or show a preview of the resulting term. The choices operation suggests ways to fill a hole in the term:

$$\begin{aligned} \mathsf{choices}(E[?_\tau]) &= \{\ \iota \mapsto E[e] \mid \forall e\ .\ \mathsf{vars}(E[?_\tau]) \vdash \tau \Rightarrow e_i \} \\ \mathsf{choose}(e) &= e \end{aligned}$$

The choices operation synthesizes possible terms of a type required by the hole. Since the state σ is a term, the choose operation simply returns it. The expression completion mode of the system behaves similarly to Idris. It constructs the term, but does not record the completion choices. A system akin to Coq that constructs a sequence of tactics could be modelled too if the state was a sequence of tactics and choices appended the available tactics to the end of the current sequence.

4.5 Structure editors

Structure editors implement a broad range of patterns of interaction for program construction. They allows us to demonstrate some of the design considerations for choose-your-own-adventure systems. We contrast a simple choose-your-own-adventure system derived from a context-free grammar, inspired by Sandblocks [7], with a system that extends the basic grammar-directed program construction with supports refactorings, akin to Deuce [18].

Formal model. We model a system that, similarly to Sandblocks, generates choices automatically based on a given context-free grammar. We assume the grammar $(\mathcal{N}, \Sigma, \mathcal{P}, S)$ consists of set of non-terminal symbols $\mathcal{N}$, terminal symbols Σ, start symbol $S \in \mathcal{N}$ and production rules $\mathcal{P}$ of the form $n \mapsto \boldsymbol{s}$ where $n \in \mathcal{N}$ and $\boldsymbol{s} \in (\mathcal{N} \cup \Sigma)^*$ (we write n for non-terminals, t for terminals and s for all symbols; bold indicates a sequence of symbols).

When interacting with the choose-your-own-adventure system, the user is gradually constructing a program (sentence) of a form $\boldsymbol{s} \in (\mathcal{N} \cup \Sigma)^*$. The system state is the program (sentence) constructed so far, i.e., $\sigma = \boldsymbol{s}$. The process can continue as long as there are non-terminal symbols in the sentence and production rules applicable to them.

In general, the choices operation could recommend applicable production rules for any non-terminal in the sentence. The following always chooses the first non-terminal by assuming a sentence $\boldsymbol{tns}$ consisting of zero or more terminals, followed by a single non-terminal and a sequence of arbitrary symbols (the need to sequentialize the construction is a limitation discussed in Section 7). The choices operation then finds all applicable production rules and offers them as choices. In a real-world system, identifiers ι_i are formed by the names of the rules (omitted from our formalism).

$$\begin{aligned} \mathsf{choices}(\boldsymbol{tns}) &= \{\ \iota_i \mapsto \boldsymbol{ts_i s} \mid \forall (n \mapsto \boldsymbol{s_i}) \in \mathcal{P}\ \} \\ \mathsf{choose}(\boldsymbol{s}) &= \boldsymbol{s} \end{aligned}$$

Unlike in the case of data exploration, the choose operation does not ensure that a program is fully constructed and the returned sentence may contain further non-terminals.

Program refactorings. The formal model derived from a context-free grammar is an adequate model for structure editors such as Sandblocks [7], but many structure editors offer more advanced operations for program construction. For example Deuce [18], which is a structure editor for a programming environment for creating Scalable Vector Graphics (SVG) images, makes it possible to transform program through a range of refactorings such as extract function, introduce a local variable, inline definition, reorder arguments and more.

Figure 7 illustrates program construction via refactoring using a simple example. It defines a small language with a grammar. Figure 7 (b) shows some of the steps necessary to construct a simple program via the grammar rules. Figure 7 (c) shows a more appealing alternative where the programmer starts with a simple program and then gradually refines it through a sequence of refactorings, making it more general.

Modelling refactorings formally requires a more expressive framework [48], so we omit the details. The choices

(a) Grammar of a minimal language with declarations and expressions

$$\begin{aligned} \mathit{expr} &:= \mathit{literal} \mid \mathit{expr} + \mathit{expr} \mid \mathit{ident} \mid \mathit{expr}(\mathit{expr}^*) \\ \mathit{decl} &:= \textbf{function}\ \mathit{ident}(\mathit{ident}^*)\{\mathit{block}\} \mid \textbf{let}\ \mathit{ident} = \mathit{expr} \\ \mathit{stmt} &:= \mathit{expr} \mid \mathit{decl} \\ \mathit{block} &:= \mathit{stmt}^* \end{aligned}$$

(b) Construction based on the grammar

1. Add *decl* as the first *stmt* and fill two *ident* leafs (name and parameters)

```
function sayHello(who) {
  stmt*
}
stmt*
```

2. Sequentiall fill holes with invocations

```
function sayHello(who) {
  print("Hello " + who)
}
sayHello("world")
```

(c) Alternat

1. Start
   prin
2. Extra
   let
   prin
3. Extra
   fun
   p
   }
   say

Figure 7. Comparison of a naive grammar-based program construction (b) and more user-friendly program construction through a sequence of refactorings (c).

operation would need to identify applicable refactorings for the current program and offer them to the user.

An important point illustrated by the two examples is whether the expression completion implemented by the two sketched choose-your-own-adventure systems is reconstructible. For a system based on non-ambiguous context-free grammar, this is the case. Although the constructed expression does not directly contain an encoding of the sequence of operations, it is possible to reconstruct which production rules have been applied (by parsing the expression according to the grammar). A choose-your-own-adventure structure editor that supports refactorings no longer has this property, because it always offers multiple ways of constructing a given program (a program can be constructed with or without a refactoring, or by using multiple refactorings in different orders).

5 Properties

The choose-your-own-adventure calculus lets us precisely compare how how**TODO: typo: duplicate "how"** different programming systems interact with the user. We saw this in Section 3, which defines reconstructible expression completion to distinguish between systems where the interaction leaves a reconstructible trace in the constructed program and systems where it does not. In this section, we make precise two properties that were introduced informally in the context of data exploration in The Gamma [37] and an additional property related to reconstruction of interaction traces.

The choose-your-own-adventure system for data exploration in The Gamma is *correct*, meaning that all programs that a user can construct using the system, by repeatedly choosing from the auto-completion list, are well-typed. The system is also *complete*, meaning that the user can use auto-completion to construct all possible programs. In other words, there are no well-typed programs that cannot be constructed interactively, by repeatedly choosing options from the offered list of choices.

Correctness. The notions of correctness and completeness can be defined for any choose-your-own-adventure systems with respect to some system-specific distinction between correct and incorrect expressions. We write $\mathcal{E} \subseteq E$ for the subset of correct expressions.

For systems based on statically-typed programming languages, a reasonable choice of $\mathcal{E}$ is a set of all well-typed expressions. For some systems, we may additionally want the set of correct expressions $\mathcal{E}$ to be hole-free, i.e., only programs that can run (or represent complete proofs) are correct. For systems where the completion is based on a grammar, we may require that correct programs do not contain non-terminals.

Definition 5.1 (Correctness)**.** **TODO: σ_0 is free: add $\forall\sigma_0$ to bind the initial state. Also motivate: what goes wrong in practice if a system lacks this property?** Assume that $\mathcal{E} \subseteq E$ is a subset of correct expressions. A choose-your-own-adventure system is correct with respect to $\mathcal{E}$ if and only if:

- $\forall\sigma_1, .., \sigma_n$ and $\iota_i, .., \iota_n$ such that $\iota_i \mapsto \sigma_i \in \text{choices}(\sigma_{i-1})$ it is the case that $\text{choose}(\sigma_i) \in \mathcal{E}$.

The definition states that, if we make any sequence of choices that start from an initial state σ_0 and result in intermediate states $\sigma_1, \ldots, \sigma_n$ then the programs we could generate from any of the intermediate states are correct.

The property depends on what we choose as the subset of correct expressions $\mathcal{E}$. Trivially, all systems are correct with respect to $\mathcal{E} = E$. However, the three of the five systems discussed in Section 4 are also correct with respect to a non-trivial set of correct expressions:

- For the data exploration system discussed in Section 4.1, we say that correct expressions are those where the parameters of all operations are fully-specified. That is, no operation requires further arguments. With respect to this definition, the system is correct. However, this is the case only because the choose operation drops the last operation if it is not fully-specified. If choose returned all operations, including the partially constructed (but not yet completed) operation, the system would not be correct.
- For type providers (Section 4.2), correct expressions are those that are well-typed. With respect to this definition, the system is correct because the choices operation offers available members based on the type information. This reasoning also applies to the type provider behind The Gamma. In The Gamma, the generated members collect operation parameters and only invoke the operation once all parameters are known.
- In the case of AI assistants (Section 4.3) the correctness of the system depends on the expressions returned by the optimization algorithm (*arg max*) from the set of all possible cleaning scripts $E_{\boldsymbol{c}}$. In general, the algorithm can return any $e \in E_{\boldsymbol{c}}$ and so system correctness is a matter of definition. The system is correct if and only if $E_{\boldsymbol{c}} \subseteq \mathcal{E}$ for all possible sets of constraints $\boldsymbol{c}$. In practice, it is more important that the constraints generated by choices are well-formed.
- For the interactive system based on type-directed synthesis (Section 4.4), correct expressions are those that are well-typed. The system is correct if the synthesis rules are sound [33], that is if $\Gamma \vdash \tau \Rightarrow e$ then also $\Gamma \vdash e : \tau$. Note that a valid choose-your-own-adventure system can be defined even using unsound synthesis rules – it would be sufficient to filter the recommended expressions in choices to the ones that are well-typed.
- A structure editor based on context-free grammar (Section 4.5) is not correct if we define $\mathcal{E}$ as the set of sentences that do not contain non-terminals. A correct structure editor could be implemented if it was based solely on refactorings such as those in Figure 7 (c).

There may be other useful systems that violate the correctness property. A tool based on a large language model (LLM) may generate code with errors that the programmer can later correct. A more interesting case would be a data exploration system, like the one discussed above, where programs only become correct after multiple subsequent choices are made, for example to fully specify arguments of an operation.

Eventual correctness. The data exploration system discussed in Section 4.1 ensures correctness by dropping the last non-fully-specified operation in the choose operation. As a result, it is correct, but it does not support reconstructible expression completion. If the operation is dropped, we cannot reconstruct the sequence of interactions from the source code. Generating code that includes the non-fully-specified operation allows reconstructibility, but makes the system incorrect. It would still satisfy a weaker eventual correctness property:

Definition 5.2 (Eventual correctness)**.** Assume that $\mathcal{E} \subseteq E$ is a subset of correct expressions, a choose-your-own-adventure system is eventually correct with respect to $\mathcal{E}$ if:

- For any sequence $\sigma_1, \ldots, \sigma_k$ and $\iota_1, \ldots, \iota_k$ such that $\forall i \in 1 \ldots k \,.\, \iota_i \mapsto \sigma_i \in \mathsf{choices}(\sigma_{i-1})$ there exists an extension $\sigma_{k+1}, \ldots, \sigma_n$ and $\iota_{k+1}, \ldots, \iota_n$ such that $\mathsf{choose}(\sigma_n) \in \mathcal{E}$ and $\forall i \in k{+}1 \ldots n \,.\, \iota_i \mapsto \sigma_i \in \mathsf{choices}(\sigma_{i-1})$.

Eventual correctness models systems where some sequences of choices result in invalid programs, but it is always possible to reach a valid program. This includes structure editors based on context-free grammars, as well as the data exploration system. Interestingly, it is always possible to turn an eventually correct system into a correct one:

1. As in the case of the data exploration, the system can remember the last state for which the choose operation returned a correct program and use it in choose until the next correct state is reached. This makes any eventually correct choose-your-own-adventure system correct, but it breaks reconstructibility of expression completion.
2. Alternatively, we can construct a system that collapses all sequence of temporarilly**TODO: typo: temporarilly→tempora** invalid states $\sigma_1, \ldots, \sigma_n$ identified by $\iota_1, \ldots, \sigma_n$**TODO: typo: last σ_n should be ι_n** where $\forall i \in 1 \ldots n{-}1 \,.\, \mathsf{choose}(\sigma_i) \notin \mathcal{E}$ and $\mathsf{choose}(\sigma_n) \in \mathcal{E}$ into a single option $\iota' \mapsto \sigma_n$ where ι' is produced by concatenating identifiers $\iota_1, \ldots, \iota_n$. This makes the system correct and also preserves reconstructibility, but it potentially generates too many choices that are difficult to navigate.

TODO: Add non-trivial theorem: prove that each of the two constructions above actually satisfies the Correctness definition.

There is more to be said about correctness of interactive programming systems. The conceptual framework provided by the choose-your-own-adventure calculus makes it possible to take the first step. Similarly, the model lets us formally define completeness.

Completeness. The programming language used in The Gamma allows users to write scripts that also use let bindings and method calls. However, for chains of member accesses which the user can construct using a type provider, it is possible to construct any chain just by repeatedly choosing options from the offered list. This is captured as the completeness property of a choose-your-own-adventure system.

Definition 5.3 (Completeness). **TODO: σ_0 is free: add $\exists\sigma_0$ to bind the initial state. Also motivate: what can't users construct if completeness is violated?** Assume that $\mathcal{E} \subseteq E$ is a subset of correct expressions. A choose-your-own-adventure system is complete with respect to $\mathcal{E}$ if and only if:

- $\forall e \in \mathcal{E} \,.\, \exists \sigma_1, .., \sigma_n$ and $\iota_1, .., \iota_n$ such that $\iota_i \mapsto \sigma_i \in \text{choices}(\sigma_{i-1})$ and $e = \text{choose}(\sigma_n)$.

A system is complete if, for any correct program, there is a sequence of choices that can be used to construct the given program. This is a more subtle property than correctness. It also does not hold for all the examples dicusssed**TODO: typo: dicusssed→discussed** in Section 4.

- The data exploration system described in Section 4.1 would be complete only if the underlying query language had a fixed set of operations, a fixed set of aggregation operations (rather than letting users write their own) and a fixed set of values for each parameter (sorting by a key, but not based on a custom expression). A completion system for a more general-purpose query language, such as SQL, would be incomplete.
- For type providers (Section 4.2), the completion mechanism is complete, because it offers all available members of the type. Consequently, the type provider in The Gamma is also complete. Even if the underlying query language is more expressive, it is hidden from the user and the system offers all available members.
- For AI assistants (Section 4.3), the system offers a set of constraints based on the current inferred program. It does not let the user construct arbitrary constraints. Moreover, because the arg max operation used in choose performs statistical optimization, there is no guarantee that it can be used to generate a specific program. The system is only complete if it is possible to choose a constraint set $\boldsymbol{c}$ that restrict the set of programs $E_{\boldsymbol{c}}$ to a single given program. This is the case for some AI assistants, including datadiff, which always offers constraints to map column to any chosen other column.
- A type-directed synthesis system (Section 4.4), or an interactive theorem prover could offer all possible ways of filling a hole with an expression containing further holes as sub-expressions. This would make the system complete (up to renaming of variables this may introduce), but the great number of generated options would be impractical. A realistic system would only generate a subset of the most useful proof/program steps and let the user write other steps manually (interactive proof construction in Idris can be seen as operating in this way).
- Completeness is easy to show for structure editors based on context-free grammars (Section 4.5). For each sentence that can be produced by the rules of the grammar, there is a sequence of choices that applies the necessary rules. However, editors based on refactorings may not have the property as some programs may not be reachable just through refactoring transformations.

Correctness and completeness are arguably both desirable properties of a choose-your-own-adventure system. Unlike for example type soundess**TODO: typo: soundess→soundness**, they are not strictly necessary in practice and are best seen as design trade-offs that designers should consider.

Uniqueness. The grammar-based structure editor example raised an interesting question about reconstructing interaction steps. Although the generated program does not directly have a sequential structure, it was possible to reconstruct the interaction steps from the expression (by parsing it and finding corresponding production rules). This is no longer possible if the program can be transformed through richer set of refactorings. The difference between the two examples is that the former has the uniqueness property:

Definition 5.4 (Uniqueness). **TODO: Bug: both paths use index n, so paths of different lengths to the same e are outside the definition. Use separate indices n and m; conclusion should be $n = m$ and $\forall i \in 1..n.\, \iota_i = \iota'_i$. Also motivate: why does the user care?** A choose-your-own-adventure system has the uniqueness property if and only if, for all expressions $e \in E$ it is the case that:

- if there are $\sigma_1, .., \sigma_n$ and $\iota_i, .., \iota_n$ such that $\iota_i \mapsto \sigma_i \in \text{choices}(\sigma_{i-1})$ and $e = \text{choose}(\sigma_n)$,
- and there are $\sigma'_1, .., \sigma'_n$ and $\iota'_i, .., \iota'_n$ such that $\iota'_i \mapsto \sigma'_i \in \text{choices}(\sigma'_{i-1})$ and $e = \text{choose}(\sigma'_n)$,
- then $\forall i \in 1 \ldots n\, .\, \iota_i = \iota'_i \wedge \sigma_i = \sigma'_i$.

The uniqueness property guarantees that, for any program, there is only one way of constructing it. This is the case for the data exploration environment (Section 4.1) and type providers (Section 4.2) where the program directly mirrors the interactive steps. It is not the case for AI assistants (Section 4.3), where adding an irrelevant constraint may have no effect on the resulting program.

A type-directed synthesis system or an interactive theorem prover (Section 4.4) would have the property if no two tactics produce the same term. This is the case for our small example, but it may not hold in general. Finally, a structure editor based on an unambiguous context-free grammar (Section 4.5) satisfies uniqueness, but a structure editor that allows further refactorings does not, in general, does not.**TODO: typo: duplicate "does not"; delete the second one**

In theory, uniqueness guarantees that we can define the decode operation required by reconstructible expression completion. However, uniqueness does not guarantee that there is a computationally effective way. To decode an expression efficiently, we need to be able to start from the initial state and identify the correct choice in each step. This is possible for a structure editor based on a context-free grammar. As a counter-example, consider a system that offers a large tree of choices and produces expression that is a hash of the choices made. For such system, there is only one path to each hash, but reconstructing it requires searching the entire tree. To guarantee a computationally effective reconstructibility, we thus need more than uniqueness – in particular, it needs to be possible to uniquely determine what part of the constructed program corresponds to which of the choices from the sequence.

theorem sum-z-rh: **forall** n **exists** $n + (\mathsf{z}) = n$.
theorem sum-s-rh: **forall** $d_1 : n_1 + n_2 = n_3$ **exists** $n_1 + (\mathsf{s}\ n_2) = (\mathsf{s}\ n_3)$.

theorem sum-commutes: **forall** $d_1 : n_1 + n_2 = n_3$ **exists** $n_2 + n_1 = n_3$
$d_2 : n_2 + n_1 = n_3$ **by induction on** d_1:
case rule

$$\frac{}{dzc:\ (\mathsf{z}) + n = n}\ \text{sum-z}$$

is
$dz_1 : n + (\mathsf{z}) = n$ **by theorem** sum-z-rh **on** n
end case
case rule

$$\frac{dsp\colon n'_1 + n_2 = n'_3}{dsc\colon (\mathsf{s}\ n'_1) + n_2 = (\mathsf{s}\ n'_3)}\ \text{sum-s}$$

is
$ds_1 : n_2 + n'_1 = n'_3$ **by induction hypothesis on** dsp
$ds_2 : n_2 + (\mathsf{s}n'_1) = (\mathsf{s}n'_3)$ **by theorem** sum-s-rh **on** ds_1
end case
end induction
end theorem

Figure 8. A proof of commutativity of + in Peano arithmetic constructed using mixed-initiative interaction. Parts generated by the system are highlighted with gray background. After the user writes a theorem and specifies induction, the system completes the first case. User then specifies how to apply the induction hypothesis and the system completes the proof.

6 Applications

The choose-your-own-adventure calculus lets us treat a wide range of interactive programming systems as instances of the same general pattern. This makes it possible to discuss properties of the systems, reuse components in system implementations, but also transfer ideas across different domains. In this section, we discuss three ideas that emerged in the context of a specific interactive system, but could be applied to other systems based on the pattern.

6.1 Mixed-initiative interaction

When using a conventional interactive theorem prover, one typically constructs the proof manually until an automated strategy can fill in the remaining gaps. This is the case with Idris proof search, as well as Coq `auto` tactics. Richer ways of interacting exist [24], but are less common. We developed a prototype mixed-initiative theorem prover [report citation omitted] that supports a more collaborative way of working where the system completes some steps automatically, but defers back to the user when it gets stuck.

Mixed-initiative theorem proving. As an illustration of the mixed-initiative proving, consider Figure 8 which shows a proof of commutativity of + in Peano arithmetic. The figure shows a version of our example, adapted from SASyLF [2, 3]

```
You are helping user to complete a task in an
interactive programming environment. The user's query
is: "Give me the athlete with the largest number of
gold medals."

The query built so far is: "olympics"."group data"."by
Athlete".

The environment offers the user possible options.
Choose an option that the should be applied to the
current dataset:

1.  count distinct Athlete
2.  count distinct Discipline
    [multiple options omitted]
13. count all
14. concatenate Athlete
15. concatenate Discipline
    [multiple options omitted]
23. sum Gold
24. sum Silver
25. sum Year

You should answer with the number of the option and no
                  further explanation.
```

Figure 9. A prompt to complete a data exploration query based on a natural language question, which uses LLM to choose from the available completion options.

and simplified for brevity. After writing proofs of sum-z-rh and sum-s-rh (not shown), the user states sum-commutes and specifies the structure of the induction. They then invoke the automatic search, which completes the first case, but gets stuck in the second case, because it fails to apply the induction hypothesis (in our full example, the failure is more subtle). The user then specifies how to apply the induction hypothesis and the system automatically completes the proof.

The interaction can be revisited from the perspective of the choose-your-own-adventure system discussed in Section 4.4. An interactive theorem prover generates possible completions using available tactics and offers them to the user, who chooses a tactic and applies it to transform the proof. In the automatic mode, the interactive theorem prover recursively searches through the available choices. If it finds one that results in a complete proof, it stops. Otherwise, it completes a number of steps (determined by some heuristic) and defers back to the user who chooses the next step and completes the proof manually or invokes the automatic search again.

Generalised mixed-initiative interaction. The mixed-initiative mode of interaction combines manual interaction with automatic search. In order to support automatic search, the system needs a metric that determines whether a constructed program is correct (as when proving a given theorem) or whether it is an improvement over another equivalent program (for example when refactoring). This general way of working can be used in other interactive programming systems:

- A system for type-directed program synthesis could automatically synthesize parts of a program (e.g., pattern matching and implementation for some of the cases), but ask user for input in order to complete branches where solution was not found automatically.
- In data exploration, a system could perform automatic search through the available operations in order to transform data into a more regular format, according to a suitability metric as done, for example, in the Proactive Wrangling system [17].

We can understand how mixed-initiative interactive systems as extensions built on top of the choose-your-own-adventure calculus. A mixed-initiative interactive system defines an operation suggest that recommends a sequence of choices for a given state.

Definition 6.1 (Mixed-initiative system)**.** A choose-your-own-adventure system supports mixed-initiative mode of interaction if it is equipped with an operation suggest such that:

- $\mathsf{suggest}(\sigma_0) = \iota_1, \ldots, \iota_n$ such that $\forall i \in 1 \ldots n \;.\; \iota_i \mapsto \sigma_i \in \mathsf{choices}(\sigma_{i-1})$.

The definition only requires that the suggested sequence of choices is valid, but it does not specify how exactly the system should make the recommendations. This is specific to a particular system. An interactive theorem prover will try to find a proof or solve sub-goals, whereas data wrangling system may try to improve the structure of data. In practice, the suggestion should improve the quality of the program so that $\mathsf{choose}(\sigma_n)$ is better than $\mathsf{choose}(\sigma_0)$, but we leave the definition flexible to accommodate a broader range of uses.

6.2 Language model-based completion

TODO: Acknowledge LLM distinction (Review B): LLM-based choices are probabilistic and may be unsound — note that this is captured by the Correctness property (LLM completions violate it) and that the model accommodates non-deterministic choices. TODO: Rewrite experiment description: new models are Claude Haiku 4.5 and Opus 4.7; 61 snippets across 5 providers (665 steps); two conditions (no-prompt baseline vs. navigation-rules system prompt); replace Figure 8; update key finding (prompt helps both models, Opus gains more: 60%→79%). Large language models can assist with data exploration by generating snippets of code based on natural language description of the problem [56]. A recognized drawback of this approach is that the user may gain only cursory understanding of the code and overlook errors. Various systems address this by generating explanations alongside with code [31]. We developed a system that provides LLM-based

natural language assistance for The Gamma [report citation omitted] based on a mechanism that assists the user and is also applicable to other choose-your-own-adventure systems.

LLM assistant for The Gamma. Our system [reprot citation omitted] integrates a type provider like the one used in The Gamma with a large language model (GPT, Gemini, DeepSeek). It lets user ask a natural language query and then recommends choices that the user should make in order to answer the query. We do not use the LLM to generate code, but to recommend an option offered by the type provider.

We construct a prompt as shown in Figure 9, which asks the LLM to recommend the next choice. In this example, the LLM is able to reliably respond with "23", which is the correct choice. Our system then pre-selects it in the completion list offered to the user. In contrast to one-shot generation of Python or SQL code snippets, our approach guides the user through the program construction, providing them with an opportunity to review and check if the program logic matches their expectations.

In addition to the basic prompting strategy, we evaluated a strategy where the LLM is provided with lookahead information, i.e., for each of the choices, we also include a list of the choices that will available subsequently (formatted as either inline information in parentheses or as a nested bullet-point list). We evaluated the sytem using queries about the Eurostat database with 10 queries for which we manually determine the correct choices. To compare the results, we compute a score for each query by following the correct path, asking for an LLM recommendation in each step and computing the ratio of correct choices. Figure 10 shows the average scores for five different strategies, using four different LLM engines.

The results suggest that, when using a more advanced LLM system, the ratio of correct recommendations is high-enough to be practically useful. Interestingly, providing more information to the LLM through the lookahead mechanism improves the quality for GPT-based systems, but not for other evaluated LLMs. Arguably, the mode of interaction where the user is offered a recommendation is preferrable over providing an opaque solution, because it keeps the human in the loop [44] and avoids "ironies of automation" [6] including deskilling.

Generalised LLM assistant. The LLM-based recommendation engine developed for The Gamma can be implemented for any choose-your-own-adventure system. As can be seen in Figure 9, the information needed to construct an LLM prompt comprise only the natural language query entered by the user, a sequence of previously made choices $\iota_1, \ldots, \iota_{n-1}$ and the identifiers $\iota_n, \iota'_n, \iota''_n, \ldots$ of the choices offered by the choices operation. As the LLM-based recommendations are based on natural language analysis of the prompt, the quality of the recommendations depends on how semantically meaningful the generated identifiers are.

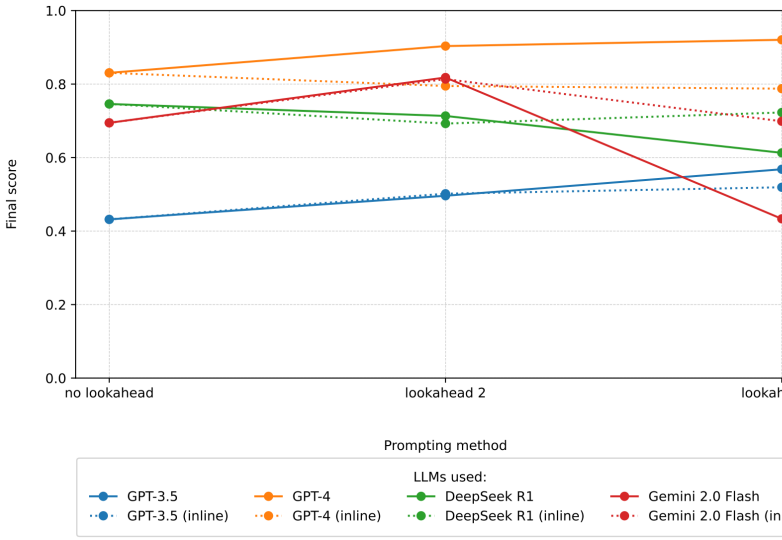

Figure 10. A plot showing the average scores for four different large language models solving 10 challenge problems using five different prompting strategies.

An LLM-based recommendation engine can be useful for a number of the choose-your-own-adventure interactive systems discussed in this paper:

- We demonstrated the usefulness of the system for data exploration in The Gamma. Type providers for structured data [39] typically generate small number of choices that the user can navigate without assistance, but navigating the schemas generated by type providers for semantic knowledge bases [50] may be simplified through a natural language query.
- The datadiff AI assistant discussed in Section 2 matches columns based on statistical distribution and ignores column names. An LLM-based recommendation engine is useful in this case, because it is able to recommend the option Don't match 'Urban.rural' and 'Nation' based solely on its name. For other AI assistants, the LLM-based recommendation engine would need access to the input data in order to be useful.
- The problem of predicting steps in order to construct a proof is known as proofstep generation in literature focused on deep learning for theorem proving [23]. Although most systems use models trained specifically for the task, there is also interest in using general-purpose LLMs [57]. Our approach suggests a potential prompting strategy.

It is interesting to note that the system sketched here combines a symbolic component (used for generating possible choices and checking their correctness) with an AI-based component (used for making choices). This is the same architecture that has been used by the AlphaGeometry system [53] for solving geometry problems.

Figure 11. Programming by demonstration in Histogram – (1) the user constructs program to load data, (2) then they filter data using graphical interface, which (3) records an operation corresponding to the interaction in code.

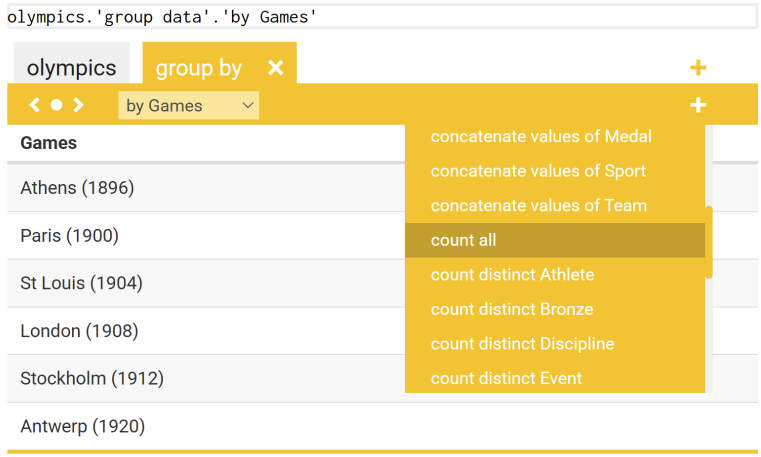

Figure 12. Specifying aggregation in The Gamma through a live preview. Grouping keys can be selected from a drop-down, while aggregations have to be added through a menu.

6.3 Direct manipulation

The Histogram programming environment [35] provides a code completion mechanism that can be used both to choose operations (as with type providers) and to specify their parameters (by offering available values in context as possible arguments). It makes it possible to evaluate sub-expressions at a fine-grained level and also refines type information based on runtime values. However, it also supports programming by demonstration [13, 21].

As illustrated in Figure 11, when the user loads data, they can manipulate it in a table through a direct manipulation interface [45]. In Histogram, interacting with the table triggers a sequence of operations that construct code. A similar functionality is available in The Gamma, where operations for manipulating tabular data can be selected not only through auto-completion, but also by interacting with the live preview.

The choose-your-own-adventure calculus enables a more general perspective on the programming by demonstration interaction as implemented by Histogram. When interacting with a live preview, the user is using a graphical interface to choose an option offered by choices. The operation appends an item to the sequence of choices, updates the system state and generates a new preview where the choice is reflected (e.g., by filtering the table).

An interesting question is whether a program can be constructed solely through direct manipulation, by interacting with the live previews. For this to be possible, the live preview needs to offer links to all the options offered by choices. This requires a careful design of live previews. In Histogram (Figure 11), the live preview offers interactive elements for filtering based on equality test (row of drop-downs), sorting (up/down arrows) and indexing (indices in the "#" column), but some operations (such as aggregation) can only be constructed through code. Previews in The Gamma provide interactive elements for all options, but those cannot always be integrated with the data display. For example, after choosing a grouping key (Figure 12), the preview shows the key column, but other columns are hidden. Specifying an aggregation requires an additional "+" button that provides access to further choices.

We can use the choose-your-own-adventure calculus to make the question whether all programs can be constructed through direct manipulation more precise. Assume $\mathsf{preview}(e) = p$ is a preview constructed for an expression e and $\mathsf{links}(p) = \iota_1, \ldots, \iota_n$ are identifiers that can be invoked through the preview (akin to links in a hypertext document).

Definition 6.2 (Direct manipulation)**.** A choose-your-own-adventure system with previews defined by preview and links supports full direct manipulation if:

- $\forall \sigma, e$ such that $\mathsf{choose}(\sigma) = e$ it is the case that $\forall \iota' \mapsto \sigma' \in \mathsf{choices}(\sigma) \,.\, \iota' \in \mathsf{links}(\mathsf{preview}(e))$.

A system supports full direct manipulation if any program can be constructed by interacting with the live previews, created based on the gradually constructed programs. As illustrated by The Gamma, this can always be achieved by listing the options in a menu, but it is more interesting to see if the links can be directly embedded in the preview as in the data table interface of Histogram.

TODO: Rewrite ARK/Morphic paragraph (Brian's email): these systems do NOT fit CYOA because interactive state is never reified into code — there is no $\mathsf{choose}(\sigma)$**. Acknowledge this as a genuine boundary of the calculus, not a future opportunity. The "opposite" of the theorem-prover case: only states, no separate expressions, making the key properties vacuous.**

The idea of directly mapping elements in a user interface to underlying structure of the programming system has previously been implemented in pioneering user interface systems including the Alternate Reality Kit [46] and the Morphic framework for Self and Squeak [26, 27]. In those systems, user interface interactions directly map to messages sent to the underlying object and the halo element in Morphic plays a similar role to the additional menu in The Gamma. It remains to be seen if the choose-your-own-adventure calculus can provide a new way of looking at those systems.

7 Limitations

TODO: Add parametric/feature-based CAD (e.g. OnShape) as an additional example domain to counter the "anecdotal" criticism: constructing a model by choosing operations from a history tree fits the calculus directly.

Arguably, the study of interactive programming systems is less well-developed than the study of programming languages. Our work is a step towards remedying of the situation, but it is only one step. As all small formal models, our system ignores a number of practical concerns.

Revisiting earlier choices. Programmers often revise programs they wrote previously. For example, a data scientist may want to add further aggregation to a grouping or change the sorting key. The choose-your-own-adventure calculus does not model how such modifications are done. Modifying an earlier choice thus poses an interesting problem – if an earlier choice is changed, we may try to "replay" the remaining choices based on the identifiers ι, but there is no guarantee that they will be offered again and that this will yield the desired result.

Sequentialization of completion. In a number of examples, an expression contained multiple holes or non-terminals that can all be completed using the choose-your-own-adventure interaction. We generally assume that they can be completed sequentially, but this may be inconvenient limitation. It may be desirable to define a variant of expression completion where multiple independent recommendations are available for different parts of the program.

Searching through options. In some systems, the number of options to choose from may be large or infinite. This can be the case for complex systems, as well as for correct systems generated from eventually correct systems. Similarly, systems may generate a very long chain of small number of options. In those cases, choosing options from a menu may be inconvenient – an alternative interface may lets the user search the tree of options.

Richer user interfaces. The choose-your-own-adventure calculus models a simple interaction based on menus. Although we point out that it can be used as the basis for richer interfaces (illustrated by the previews in programming by demonstration), it may be interesting to look at richer interfaces. In particular, a number of systems are centred around entities that can be selected and modified through commands or interactions. This includes Morphic's halos [26, 27], the classic Alternate Reality Kit [47], as well as text-based systems [11].

8 Conclusions

Working in many interactive programming environments has the same feel as following a choose-your-own-adventure book. You start and repeatedly choose one of the offered options to construct a program, proof or to explore data. The aim of this paper is to formally capture this kind of interaction. As with formal models of programming languages, our model focuses on the minimal essence of the interaction pattern. Yet, this is sufficient to recognize similarities across a broad range of systems, talk about key properties that they may have, transfer knowledge across multiple domains, as well as to suggest a more general way of building richer programming experiences ranging from AI-assistants and mixed-initiative interaction to programming by demonstration.

References

[1] Michael D. Adams, Eric Griffis, Thomas J. Porter, Sundara Vishnu Satish, Eric Zhao, and Cyrus Omar. 2025. Grove: A Bidirectionally Typed Collaborative Structure Editor Calculus. *Proc. ACM Program. Lang.* 9, POPL, Article 73 (Jan. 2025), 29 pages. https://doi.org/10.1145/3704909

[2] Jonathan Aldrich and John Boyland. 2025. Formalization of integers, addition, and a proof that addition commutes (SASyLF example). https://github.com/boyland/sasylf/blob/master/examples/sum.slf Accessed: 2025-03-06.

[3] Jonathan Aldrich, Robert J. Simmons, and Key Shin. 2008. SASyLF: an educational proof assistant for language theory. In *Proceedings of the 2008 International Workshop on Functional and Declarative Programming in Education* (Victoria, BC, Canada) *(FDPE '08)*. Association for Computing Machinery, New York, NY, USA, 31–40. https://doi.org/10.1145/1411260.1411266

[4] Thorsten Altenkirch, Veronica Gaspes, Bengt Nordström, and Björn von Sydow. 1994. *A user's guide to ALF*. Technical Report. Chalmers University of Technology, Sweden. https://people.cs.nott.ac.uk/psztxa/publ/alf94.pdf Unpublished Draft.

[5] David Aspinall. 2000. Proof General: A Generic Tool for Proof Development. In *Tools and Algorithms for the Construction and Analysis of Systems*, Susanne Graf and Michael Schwartzbach (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 38–43.

[6] Lisanne Bainbridge. 1983. Ironies of Automation. In *Analysis, Design and Evaluation of Man–Machine Systems*, G. Johannsen and J.E. Rijnsdorp (Eds.). Pergamon, 129–135. https://doi.org/10.1016/B978-0-08-029348-6.50026-9

[7] Tom Beckmann, Patrick Rein, Stefan Ramson, Joana Bergsiek, and Robert Hirschfeld. 2023. Structured Editing for All: Deriving Usable Structured Editors from Grammars. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) *(CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 595, 16 pages. https://doi.org/10.1145/3544548.3580785

[8] Alan F. Blackwell and Thomas R. G. Green. 2003. Notational Systems – The Cognitive Dimensions of Notations Framework. In *HCI Models, Theories, and Frameworks: Toward a Multidisciplinary Science*, John M. Carroll (Ed.). Morgan Kaufmann, San Francisco, 103–134.

[9] Edwin Brady. 2015. *The Idris Programming Language.* Springer International Publishing, Cham, 115–186. https://doi.org/10.1007/978-3-319-15940-9_4

[10] Edwin Brady. 2021. Idris 2: Quantitative Type Theory in Practice. In *35th European Conference on Object-Oriented Programming (ECOOP 2021) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 194)*, Anders Møller and Manu Sridharan (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 9:1–9:26. https://doi.org/10.4230/LIPIcs.ECOOP.2021.9

[11] Omar Antolín Camarena. 2025. Embark: Emacs Mini-Buffer Actions Rooted in Keymaps. https://github.com/oantolin/embark Accessed: 2025-03-06.

[12] E. F. Codd. 1970. A relational model of data for large shared data banks. *Commun. ACM* 13, 6 (June 1970), 377–387. https://doi.org/10.1145/362384.362685

[13] A. Cypher and D.C. Halbert. 1993. *Watch what I Do: Programming by Demonstration.* MIT Press. https://books.google.cz/books?id=Ggzjo0-W1y0C

[14] Damian Frölich and L. Thomas van Binsbergen. 2024. On the Soundness of Auto-completion Services for Dynamically Typed Languages. In *Proceedings of the 23rd ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences* (Pasadena, CA, USA) *(GPCE '24).* Association for Computing Machinery, NY, USA, 107–120. https://doi.org/10.1145/3689484.3690734

[15] Richard P. Gabriel. 2012. The structure of a programming language revolution. In *Proceedings of the ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (Tucson, Arizona, USA) *(Onward! 2012).* Association for Computing Machinery, New York, NY, USA, 195–214. https://doi.org/10.1145/2384592.2384611

[16] Thomas R. G. Green. 1989. Cognitive Dimensions of Notations. In *People and Computers V: Proceedings of the Fifth Conference of the British Computer Society*, A. Sutcliffe and L. Macaulay (Eds.). Cambridge University Press, Cambridge, UK, 443–460.

[17] Philip J. Guo, Sean Kandel, Joseph M. Hellerstein, and Jeffrey Heer. 2011. Proactive wrangling: mixed-initiative end-user programming of data transformation scripts. In *Proceedings of the 24th Annual ACM Symposium on User Interface Software and Technology* (Santa Barbara, California, USA) *(UIST '11).* Association for Computing Machinery, New York, NY, USA, 65–74. https://doi.org/10.1145/2047196.2047205

[18] Brian Hempel, Justin Lubin, Grace Lu, and Ravi Chugh. 2018. Deuce: a lightweight user interface for structured editing. In *Proceedings of the 40th International Conference on Software Engineering* (Gothenburg, Sweden) *(ICSE '18).* Association for Computing Machinery, New York, NY, USA, 654–664. https://doi.org/10.1145/3180155.3180165

[19] Jack Hughes and Dominic Orchard. 2024. Program Synthesis from Graded Types. In *Programming Languages and Systems*, Stephanie Weirich (Ed.). Springer Nature Switzerland, Cham, 83–112.

[20] Joel Jakubovic, J. Edwards, and T. Petricek. 2023. Technical Dimensions of Programming Systems. *Art Sci. Eng. Program.* 7, 3 (2023). https://doi.org/10.22152/PROGRAMMING-JOURNAL.ORG/2023/7/13

[21] Sean Kandel, Andreas Paepcke, Joseph Hellerstein, and Jeffrey Heer. 2011. Wrangler: interactive visual specification of data transformation scripts. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Vancouver, BC, Canada) *(CHI '11).* Association for Computing Machinery, New York, NY, USA, 3363–3372. https://doi.org/10.1145/1978942.1979444

[22] Tristan Knoth. 2023. *Type-Directed Program Synthesis.* Ph. D. Dissertation. University of California, San Diego. https://escholarship.org/uc/item/4g11m7rq

[23] Zhaoyu Li, Jialiang Sun, Logan Murphy, Qidong Su, Zenan Li, Xian Zhang, Kaiyu Yang, and Xujie Si. 2024. A Survey on Deep Learning for Theorem Proving. In *First Conference on Language Modeling.* https://openreview.net/forum?id=zlw6AHwukB

[24] Helen Lowe and David Duncan. 1997. XBarnacle: Making theorem provers more accessible. In *Automated Deduction—CADE-14*, William McCune (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 404–407.

[25] Lena Magnusson and Bengt Nordström. 1994. The Alf proof editor and its proof engine. In *Types for Proofs and Programs*, Henk Barendregt and Tobias Nipkow (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 213–237.

[26] John Maloney. 2001. An Introduction to Morphic: The Squeak User Interface Framework. In *Squeak: Open Personal Computing and Multimedia*, Mark Guzdial and Kim Rose (Eds.). Prentice Hall.

[27] John H. Maloney and Randall B. Smith. 1995. Directness and liveness in the morphic user interface construction environment. In *Proceedings of the 8th Annual ACM Symposium on User Interface and Software Technology* (Pittsburgh, Pennsylvania, USA) *(UIST '95).* Association for Computing Machinery, New York, NY, USA, 21–28. https://doi.org/10.1145/215585.215636

[28] Mikaël Mayer, Viktor Kuncak, and Ravi Chugh. 2018. Bidirectional evaluation with direct manipulation. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 127 (Oct. 2018), 28 pages. https://doi.org/10.1145/3276497

[29] Conor McBride. 1999. *Dependently Typed Functional Programs and their Proofs.* Ph. D. Dissertation. University of Edinburgh. https://era.ed.ac.uk/handle/1842/374

[30] David Moon, Andrew Blinn, and Cyrus Omar. 2022. tylr: a tiny tile-based structure editor. In *Proceedings of the 7th ACM SIGPLAN International Workshop on Type-Driven Development* (Ljubljana, Slovenia) *(TyDe 2022).* Association for Computing Machinery, New York, NY, USA, 28–37. https://doi.org/10.1145/3546196.3550164

[31] Farhad Nooralahzadeh, Yi Zhang, Jonathan Furst, and Kurt Stockinger. 2024. Explainable Multi-Modal Data Exploration in Natural Language via LLM Agent. arXiv:2412.18428 [cs.AI] https://arxiv.org/abs/2412.18428

[32] Cyrus Omar, Ian Voysey, Ravi Chugh, and Matthew A. Hammer. 2019. Live functional programming with typed holes. *Proc. ACM Program. Lang.* 3, POPL, Article 14 (Jan. 2019), 32 pages. https://doi.org/10.1145/3290327

[33] Peter-Michael Osera and Steve Zdancewic. 2015. Type-and-example-directed program synthesis. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Portland, OR, USA) *(PLDI '15).* Association for Computing Machinery, New York, NY, USA, 619–630. https://doi.org/10.1145/2737924.2738007

[34] Tomas Petricek. 2017. Data Exploration through Dot-driven Development. In *31st European Conference on Object-Oriented Programming, ECOOP 2017, June 19-23, 2017, Barcelona, Spain (LIPIcs, Vol. 74)*, Peter Müller (Ed.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 21:1–21:27. https://doi.org/10.4230/LIPICS.ECOOP.2017.21

[35] Tomas Petricek. 2019. Histogram: You have to know the past to understand the present. In *Workshop on Live Programming (LIVE 2019).* Athens, Greece. https://tomasp.net/histogram Presented at the Workshop on Live Programming, Published online.

[36] Tomas Petricek. 2020. Foundations of a live data exploration environment. *Art Sci. Eng. Program.* 4, 3 (2020), 8. https://doi.org/10.22152/PROGRAMMING-JOURNAL.ORG/2020/4/8

[37] Tomas Petricek. 2022. The Gamma: Programmatic Data Exploration for Non-programmers. In *2022 IEEE Symposium on Visual Languages and Human-Centric Computing, VL/HCC 2022, Rome, Italy, September 12-16, 2022*, Paolo Bottoni, Gennaro Costagliola, Michelle Brachman, and Mark Minas (Eds.). IEEE, 1–7. https://doi.org/10.1109/VL/HCC53370.2022.9833134

[38] Tomas Petricek, James Geddes, and Charles Sutton. 2018. Wrattler: Reproducible, live and polyglot notebooks. In *10th USENIX Workshop on the Theory and Practice of Provenance (TaPP 2018).* USENIX Association, London. https://www.usenix.org/conference/tapp2018/presentation/

petricek

[39] Tomas Petricek, Gustavo Guerra, and Don Syme. 2016. Types from data: making structured data first-class citizens in F#. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Santa Barbara, CA, USA) *(PLDI '16)*. Association for Computing Machinery, New York, NY, USA, 477–490. https://doi.org/10.1145/2908080.2908115

[40] Tomas Petricek, Gerrit J. J. van den Burg, Alfredo Nazábal, Taha Ceritli, Ernesto Jiménez-Ruiz, and Christopher K. I. Williams. 2023. AI Assistants: A Framework for Semi-Automated Data Wrangling. *IEEE Trans. Knowl. Data Eng.* 35, 9 (2023), 9295–9306. https://doi.org/10.1109/TKDE.2022.3222538

[41] Clément Pit-Claudel and Pierre Courtieu. 2016. Company-Coq: Taking Proof General One Step Closer to a Real IDE. In *Second International Workshop on Coq for PL (CoqPL '16)*. https://dspace.mit.edu/handle/1721.1/101149.2

[42] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. 2016. Program synthesis from polymorphic refinement types. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Santa Barbara, CA, USA) *(PLDI '16)*. Association for Computing Machinery, New York, NY, USA, 522–538. https://doi.org/10.1145/2908080.2908093

[43] Mark Seemann. 2021. *Code That Fits in Your Head: Heuristics for Software Engineering*. Addison-Wesley Professional, Boston.

[44] Abigail Sellen and Eric Horvitz. 2024. The Rise of the AI Co-Pilot: Lessons for Design from Aviation and Beyond. *Commun. ACM* 67, 7 (July 2024), 18–23. https://doi.org/10.1145/3637865

[45] Ben Shneiderman. 1983. Direct Manipulation: A Step Beyond Programming Languages. *Computer* 16, 8 (1983), 57–69. https://doi.org/10.1109/MC.1983.1654471

[46] Randall B. Smith. 1986. Experiences with the alternate reality kit: an example of the tension between literalism and magic. In *Proceedings of the SIGCHI/GI Conference on Human Factors in Computing Systems and Graphics Interface* (Toronto, Ontario, Canada) *(CHI '87)*. Association for Computing Machinery, New York, NY, USA, 61–67. https://doi.org/10.1145/29933.30861

[47] Randall B. Smith. 1986. Experiences with the alternate reality kit: an example of the tension between literalism and magic. In *Proceedings of the SIGCHI/GI Conference on Human Factors in Computing Systems and Graphics Interface* (Toronto, Ontario, Canada) *(CHI '87)*. Association for Computing Machinery, New York, NY, USA, 61–67. https://doi.org/10.1145/29933.30861

[48] Friedrich Steimann, Christian Kollee, and Jens von Pilgrim. 2011. A Refactoring Constraint Language and Its Application to Eiffel. In *ECOOP 2011 – Object-Oriented Programming*, Mira Mezini (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 255–280.

[49] Charles A. Sutton, Timothy Hobson, James Geddes, and Rich Caruana. 2018. Data Diff: Interpretable, Executable Summaries of Changes in Distributions for Data Wrangling. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2279–2288. https://doi.org/10.1145/3219819.3220057

[50] Don Syme, Keith Battocchi, Kenji Takeda, Donna Malayeri, Jomo Fisher, Jack Hu, Tao Liu, Brian McNamara, Daniel Quirk, Matteo Taveggia, Wonseok Chae, Uladzimir Matsveyeu, and Tomas Petricek. 2012. *F# 3.0 - Strongly-Typed Language Support for Internet-Scale Information Sources*. Technical Report MSR-TR-2012-101. https://www.microsoft.com/en-us/research/publication/f3-0-strongly-typed-language-support-for-internet-scale-information-sources/

[51] Don Syme, Keith Battocchi, Kenji Takeda, Donna Malayeri, and Tomas Petricek. 2013. Themes in information-rich functional programming for internet-scale data sources. In *Proceedings of the 2013 Workshop on Data Driven Functional Programming, DDFP 2013, Rome, Italy, January 22, 2013*, Evelyne Viegas, Karin K. Breitman, and Judith Bishop (Eds.). ACM, 1–4. https://doi.org/10.1145/2429376.2429378

[52] Tim Teitelbaum and Thomas Reps. 1981. The Cornell program synthesizer: a syntax-directed programming environment. *Commun. ACM* 24, 9 (Sept. 1981), 563–573. https://doi.org/10.1145/358746.358755

[53] Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. 2024. Solving olympiad geometry without human demonstrations. *Nature* 625, 7995 (01 Jan 2024), 476–482. https://doi.org/10.1038/s41586-023-06747-5

[54] Jan Liam Verter and Tomas Petricek. 2024. Don't Call Us, We'll Call You: Towards Mixed-Initiative Interactive Proof Assistants for Programming Language Theory. *CoRR* abs/2409.13872 (2024). https://doi.org/10.48550/arXiv.2409.13872 arXiv:2409.13872 Presented at the 5th International Workshop on Human Aspects of Types and Reasoning Assistants (HATRA 2024).

[55] Gregor Weber. 2017. Code is not just text: Why our code editors are inadequate tools. In *Companion Proceedings of the 1st International Conference on the Art, Science, and Engineering of Programming* (Brussels, Belgium) *(Programming '17)*. Association for Computing Machinery, New York, NY, USA, Article 35, 3 pages. https://doi.org/10.1145/3079368.3079415

[56] Pengcheng Yin, Wen-Ding Li, Kefan Xiao, Abhishek Rao, Yeming Wen, Kensen Shi, Joshua Howland, Paige Bailey, Michele Catasta, Henryk Michalewski, Oleksandr Polozov, and Charles Sutton. 2023. Natural Language to Code Generation in Interactive Data Science Notebooks. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 126–173. https://doi.org/10.18653/v1/2023.acl-long.9

[57] Shizhuo Dylan Zhang, Talia Ringer, and Emily First. 2023. Getting More out of Large Language Models for Proofs. *CoRR* abs/2305.04369 (2023). https://doi.org/10.48550/arXiv.2305.04369 arXiv:2305.04369 [cs.FL] Presented at the 8th Conference on Artificial Intelligence and Theorem Proving (AITP 2023).